3. Правила проектирования и ведения документации по системному анализу

1 Оглавление

При создании используйте макрос [redacted] Оглавление

- 1 Оглавление
- 2 Общие положения
  - 2.1 История изменения
  - 2.2 Список ответственных лиц
  - 2.3. Лист согласования
  - 2.4 Общие сведения
    - 2.4.1 Термины и определения
    - 2.4.2 Используемые документы
    - 2.4.3 Общее описание изменений
    - 2.4.4 Цель доработки
    - 2.4.5 Условия и ограничения
    - 2.4.6 Критерии успеха, критерии приемки
- 2.5 Сценарии использования (UseCase)
- 2.6 Диаграммы
  - 2.6.1 Диаграммы последовательности UML
  - 2.6.2 Иные диаграммы
- 3 Функциональные требования
  - 3.1 Описание Frontend
    - 3.1.1 Спецификация на интерфейс пользователя
    - 3.1.2. Ссылки на макеты (Figma)
  - 3.2 Описание Backend
    - 3.2.1 Интерфейсы передачи информации
    - 3.2.2 Шаблон сервиса
    - 3.2.3 Модель данных. Описание БД
    - 3.2.4 Описание API
  - 3.3 Интеграции
    - 3.3.1 Внешние интеграции
    - 3.3.2 Внутренние интеграции
  - 3.4. Метрики
- 4 Нефункциональные требования
  - 4.2 Требования к производительности
  - 4.3 Требования к масштабируемости
  - 4.4 Требования к надежности
  - 4.5 Требования к сопровождению

2 Общие положения

2.1 История изменения

В начале каждого ТЗ создается скрытая таблица История изменения, куда вписываются версия, вносимые изменения, ссылки на задачи, дата изменения и автор

Версия	Ссылка на задачу Jira	Описание изменения	Автор	Дата
1.0		[redacted]	[redacted]	21 янв. 2025 г.

2.2 Список ответственных лиц

Роль	Структурное подразделение	ФИО
Заказчик		
Руководитель проекта		
Разработчик ПО		

Аналитик		
…		
Иные роли		

2.3. Лист согласования

Лист согласования - реквизит с указанием ответственных и статуса согласования. В рамках бизнес-процесса согласования могут быть выставлены замечания, требующие устранения аналитиком.

Возможные статусы документа

- требуется согласование
- в работе
- согласовано
- возвращено на доработку
- согласовано с замечаниями (приложен протокол замечаний в виде отдельной задачи или комментариев к ТЗ с указанием сроков устранения)
- не согласовано

Роль	Статус	Комментарий	ФИО	Дата
Заказчик	Согласовано			
Сотрудник подразделения ИБ	Возвращено на доработку			
…				
Иные роли				

Матрица ответственных за согласование артефактов

Артефакт документации	Архитекторы	Product Owner	Системные аналитики	Разработка	Тестирование
Бизнес- требования	-	-	Согласование	-	-
Архитектура	Согласование	-	-	-	-
UseCase, варианты использования	-	Согласование	-	Согласование	Согласование
UI в Figma	-	-	Согласование	-	-

2.4 Общие сведения

2.4.1 Термины и определения

В разделе описываются в табличной форме определения, акронимы и сокращения. Используйте в том числе общепринятые определения из Глоссария Russ [redacted]. Глоссарий и порядок оформления ссылок на якоря Глоссария по ссылке.

Желательно применять глоссарий БТ и дополнить его, если необходимо.

Термин	Определение
….	
….	

2.4.2 Используемые документы

№	Наименование документа	Версия, дата	Ссылка или файл
1	Бизнес-требования		
2	Описание архитектуры		
3	Роли и характеристика пользователей		
4	Связанные требования		

5	.....		

2.4.3 Общее описание изменений

Общее, без излишней детализации, описание изменений, которые должны быть выполнены в результате реализации настоящего ТЗ. Лучше воспользоваться соответствующими бизнес-формулировками из БТ по вашей доработке

2.4.4 Цель доработки

Описывается бизнес-цель доработки, также можно воспользоваться формулировками из БТ

2.4.5 Условия и ограничения

Описать границы доработки, условия и ограничения – как бизнес-, так и системные

2.4.6 Критерии успеха, критерии приемки

Иначе DoD (definition of done). Укажите критерии успеха, связанные с реализацией функционала, по которым можно будет судить, что ТЗ реализовано в полном объеме

2.5 Сценарии использования (UseCase)

Сценарий использования, вариант использования, прецедент использования (англ. use case) — в разработке программного обеспечения и системном проектировании это описание поведения системы, когда она взаимодействует с кем-то (или чем-то) из внешней среды. Создайте текстовое описание по шагам или схему на языке uml или используйте готовые Usecase из БТ

UseCase
Основные элементы сценария использования (Use Case): Идентификатор (ID) — уникальный номер или код для отслеживания сценария использования. Название (Title) — краткое описание цели взаимодействия пользователя с системой. Акторы, участники (Actors) — пользователи или системы, которые взаимодействуют с основной системой. Предусловия (Preconditions) — условия, которые должны быть выполнены перед началом сценария. Основной поток (Main Flow) — последовательность шагов для достижения цели. Альтернативные потоки (Alternative Flows) — шаги, которые могут быть выполнены в случае отклонений от основного потока. Исключения (Exceptions) — возможные ошибки или исключительные ситуации. Постусловия (Postconditions) — условия, которые должны быть выполнены после завершения сценария. Ожидаемый результат (Expected Result) — что должно произойти после выполнения всех шагов.

ID: UC_001

Название: Добавление комментария на странице сайта.

Акторы:

- Пользователь (User)
- Система (System)

Предусловия:

- Пользователь должен быть авторизован на сайте.
- Пользователь должен находиться на странице статьи.

Основной поток:

- Пользователь открывает страницу статьи.
- Пользователь пролистывает страницу вниз до раздела комментариев.
- Пользователь вводит текст в поле "Комментарий".
- Пользователь нажимает кнопку "Отправить".
- Система сохраняет комментарий в базе данных.
- Система отображает новый комментарий на странице статьи.

Альтернативные потоки:

Альтернативный поток 1 (AP1):

- Пользователь вводит пустой комментарий.
- Система отображает сообщение об ошибке: "Комментарий не может быть пустым"

Альтернативный поток 2 (AP2):

- Пользователь вводит комментарий, содержащий запрещенный контент.
- Система отображает сообщение об ошибке: "Комментарий содержит запрещенный контент"

Исключения:

Исключение 1 (EX1):

- Система временно недоступна.
- Система отображает сообщение об ошибке: "Сервис временно недоступен. Попробуйте позже."

Постусловия:

- Комментарий успешно добавлен и отображен на странице статьи.
- Время отправки комментария отображено корректно.

Ожидаемый результат:

- Комментарий успешно сохранен в базе данных и отображен на странице статьи.
- Пользователь получает подтверждение о том, что комментарий успешно добавлен.

UseCase можно сделать в текстовом или табличном виде. Также можно пронумеровать шаги

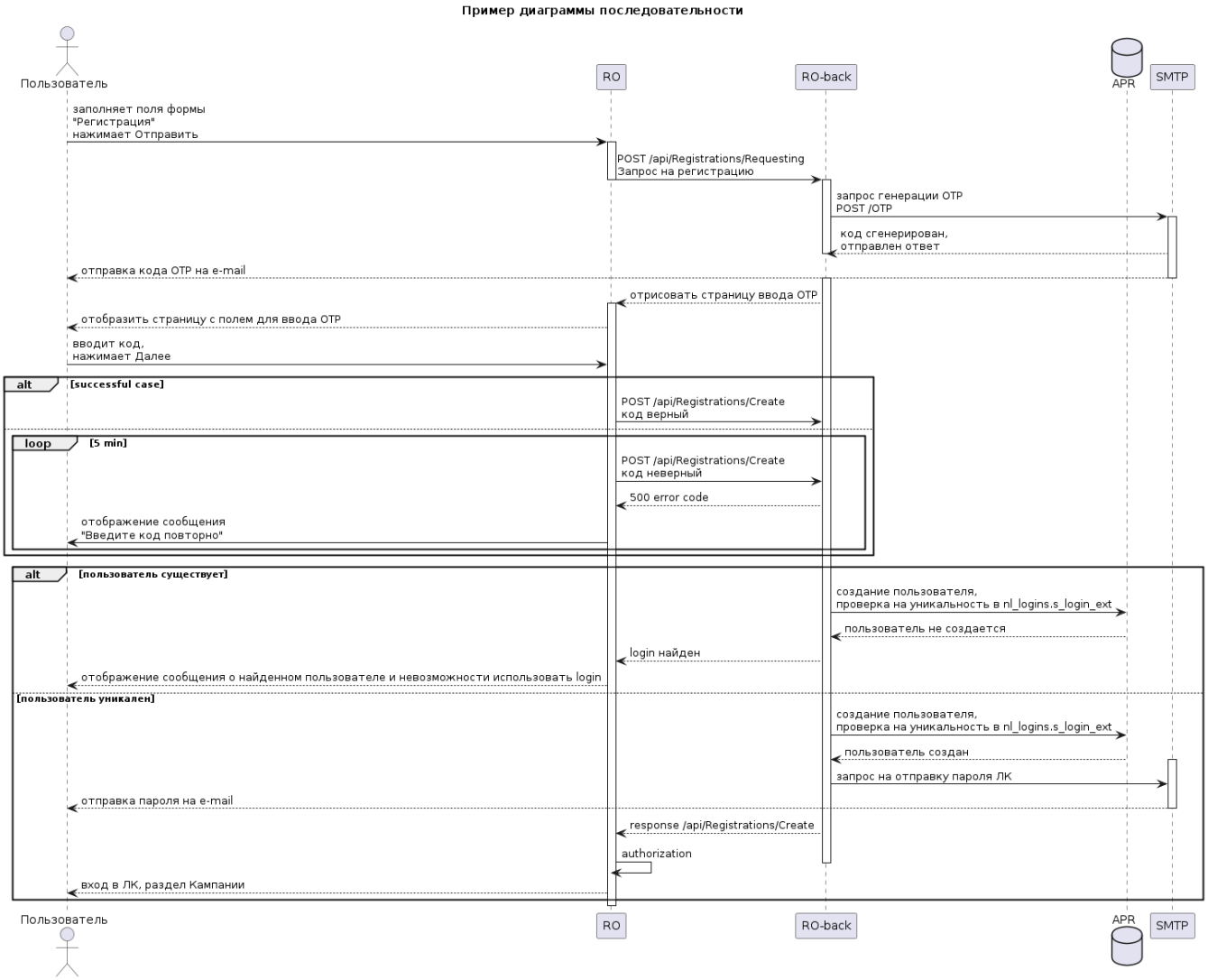
UC-1 Смена пароля пользователя

	Название	Наш пример
1	Действующие лица	Пользователь, Система
2	Цель	Сменить пароль пользователя для входа в ЛК
3	Предусловие	Пользователь уже зарегистрирован, имеет действующий логин и пароль
4	Запускающее событие	Пользователь активизирует функцию смены пароля на странице авторизации
5	Основной сценарий	1. Пользователь активирует Смену пароля. 2. Система предоставляет возможность ввода данных 3. Пользователь вводит свой логин, старый пароль, повторно старый пароль и подтверждает ввод. 4. Система сообщает об успешной смене пароля (ссылка на сообщение) и возвращает окно авторизации
6	Альтернативный сценарий	1-3 те же. 4. Система уведомляет пользователя об ошибке при вводе данных, указывая в каком поле и какой тип ошибки (ссылка) 5. Пользователь заново вводит данные и подтверждает ввод. 6. Переходим к п.4
7	Исключения	В настройках безопасности для данного IP адреса существует запрет на вход в систему. Результат: форма авторизации не предоставляется, система выдает сообщение пользователю (ссылка).
8	Постусловия	Новый логин пользователя сохранен в БД.

2.6 Диаграммы

2.6.1 Диаграммы последовательности UML

1. Для размещения диаграммы UML необходимо использовать плагин PlantUML Macro. Если доработка незначительная и не затрагивает логику последовательности вызовов, сослаться на диаграммы из предыдущих ТЗ.
2. Вставляйте диаграмму соответствующим плагином. Загружать обычные картинки - плохой тон, так как при необходимости редактирования другим сотрудником ему придется рисовать всё с нуля. **Запрещено** вставлять скриншоты из других спецификаций
3. Правила и подсказки по созданию UML будут описаны отдельно.



2.6.2 Иные диаграммы

(например BPMN)

К структурным диаграммам относят следующие 7 типов диаграмм:

- Диаграмма составной структуры
- Диаграмма развертывания
- Диаграмма пакетов
- Диаграмма профилей
- Диаграмма классов
- Диаграмма объектов
- Диаграмма компонентов - важная диаграмма, предшественница ERD на уровне сущностей

А к диаграммам поведения относят следующие типы диаграмм:

- Диаграмма деятельности
- Диаграмма прецедентов
- Диаграмма состояний
- Диаграмма последовательности
- Диаграмма коммуникаций
- Диаграмма обзора взаимодействия
- Временная диаграмма

составьте и опубликуйте при необходимости

3 Функциональные требования

Каждая функция (пункт) функциональных требований должны быть пронумерованы (например, FR-1, FR-2).

Сделайте трассировку пунктов бизнес-требований и функциональных требований (Трассировка требований - пример)

Frontend (далее FE) и backend (далее BE) могут описываться отдельно в виде спецификаций FE и BE в дочерней структуре страниц родительского ТЗ либо отдельными нумерованными разделами в общем сквозном описании

Общие правила:

1. Доработки описываются:
    a. в формате as is (текущая реализация) для уже реализованного функционала. Например: "Система работает", "Сервис запрашивает", "Производится проверка" и т.д.
    b. в формате to be (будущая реализация) для доработок. После вывода доработки на прод необходимо отредактировать формат to be - привести к as is.
    c. формат "Требуется реализовать" и пр. в том числе используется в постановках в Jira. Без описания в [redacted] продуктовая задача существовать в Jira не может, обязательна ссылка на документацию.
2. В описании Frontend нужно указывать на сервисы backend, которые FE вызывает.
3. Сервисы описываются на страницах профильных пространств. Родительская страница описывает общую информацию, дочерние - методы, логику работы метода, модель здоровья и т.д.
4. Не пренебрегайте примерами запросов и ответов и другими возможными вариантами пояснения и детализации описания. Любой большой объект в [redacted] (в том числе схему) можно скрыть через плагин "Раскрыть" или "Код". К примеру, ваш сервис вызывает два других сервиса, обрабатывает их ответ и отдает на FE. Желательно в спецификации BE, как минимум, указать пример ответа на FE или те участки ответов задействованных сервисов, которые нужны в вашем процессе.
5. Не забывайте при этом, что хорошая статья!=большая статья. Документация не должна быть избыточной, мы пишем не ради процесса, а для того, чтобы пользователям было понятно и статью было удобно использовать как в процессе разработки, так и в дальнейшей эксплуатации тем, кто хочет составить понятие о процессе.
6. Если вы ссылаетесь на какой-то другой сервис, экран и пр., то требуется делать гиперссылку на соответствующую статью. Если спецификации сервиса нет, то в крайнем случае можно сделать ссылку на swagger.
7. Если функциональность не в проде, то во введении можно писать примечания с текстом "В разработке".

    > В разработке

8. Изменение существующей спецификации, например, модели данных/контракта следует помечать цветом или заливкой для облегчения визуального восприятия разработчиками границ доработки. После реализации изменения цвет или заливка удаляются. Если что-то в спецификации удалено, то следует оставить предупреждение об удалении.
9. В таблице История изменений также следует отметить цветом задачу Jira
10. **ТЗ одного функционала составляется инкрементами, то есть в каждой новой версии в сквозном описании процесса дополняются /изменяются существующие разделы**

3.1 Описание Frontend

3.1.1 Спецификация на интерфейс пользователя

Этот пункт присутствует в ТЗ на разработку при реализации по прототипам или макетам.

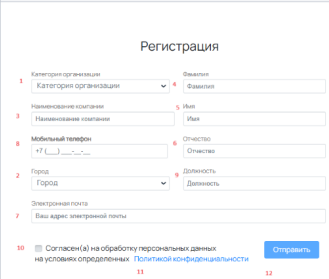
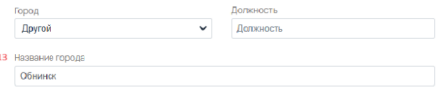
- Спецификация — это документ, который описывает функциональное назначение и тип каждого элемента управления/блока, техническую составляющую интерфейса. Как правило, спецификация составляется по типу: Элемент-Изображение-Описание.

3.1.2. Ссылки на макеты (Figma)

В дополнение к общим сведениям (раздел выше) выделяются следующие правила и рекомендации. Требуется указывать/учитывать:

1. Скриншоты, где каждый пункт помечен и указан в таблице с описанием;
2. В случае иконок и картинок, названия которых могут приходить от бэкенда, желательно описывать логику их хранения, скачивания и кэширования.
3. Скриншоты (экранные формы) должны быть помещены в таблицу вида "ЭФ - Изображение - Описание"

2.1. Экранная форма Регистрация

ЭФ	GUI	Описание
1		Экран Регистрация. Пользователь заполняет поля формы "Регистрация" Пользователь отправляет заполненные данные ("Отправить") и переходит на страницу ввода кода OTP
2		Скрываемое поле "Название города", условие отображения - Город = "Другой"

4. Должно быть столько скриншотов, чтобы можно было пошагово понять весь бизнес-процесс. Иногда макеты могут изменить свое местоположение или полностью измениться и единственным источником информации о процессе будет ваша документация
5. В таблице важно описывать хедеры, элементы управления, потому что в разных случаях один и тот же элемент может выполнять разную логику (например, нажатие кнопки "Назад" может перебрасывать или на главную страницу или на предыдущую страницу. То же касается логики закрытия уведомляющий сообщений или завершения процесса)
6. Если обработка действий пользователя подразумевает вызов метода, указать метод BE, сослаться на спецификацию в виде гиперссылки или на сваггер

11	Ссылка для просмотра соглашения о Политике конфиденциальности	-	нет	-	При клике на ссылку пользователю открывается новая вкладка браузера url https://russonline-ro-front-test.russoutdoor.ru/public/privacy-policy
12	Отправить	-		-	Кнопка задизейблена, если не поднят флаг согласия на обработку персональных данных (п.10). При клике на кнопку Отправить отправляется запрос на регистрацию POST/api/Registrations/Requesting

7. ⚠️ Обязательно в итоговой таблице указывать
   a. обязательность параметров (required), как заполняются поля ввода или какой тип элемента UI/UX использован, по возможности, привести пример заполнения
   b. маппинг полей формы фронтенда на поля метода бэкенда, правила преобразования, если необходимо, маску поля, регулярное выражение, если используется

2.1.1 Описание полей экранной формы Регистрация

Маппинг

#	Элемент экранной формы	Placeholder	Required	Поле метода	Value
1	Категория организации	Категория организации	да	regcategory	Значения справочника: • Рекламное агентство • Прямой рекламодатель
2	Город	Город	да	areaId	Значение справочника регионов.
3	Наименование компании	Наименование компании	да	companyName	Заполняет пользователь
4	Фамилия	Фамилия	да	firstName	Заполняет пользователь
5	Имя	Имя	да	lastName	Заполняет пользователь
6	Отчество	Отчество	нет	middleName	Заполняет пользователь
7	Электронная почта	Ваш адрес электронной почты	да	email	Заполняет пользователь
8	Мобильный телефон	+7(___)__ -__- __	да	phone	Заполняет пользователь Маска ввода в формате +7(___)__ -__- __
9	Должность	Должность	да	positionName	Заполняет пользователь
10	Согласен(а) на обработку персональных данных на условиях, определенных	-	да	acceptTerms	Чекбокс, заполняет пользователь

8. Для элементов управления, например, кнопки, можно использовать следующее описание
   a. Кнопка <Наименование кнопки>

Параметр	Значение	Комментарий
Отображение	• Наименование: • Видимость: • Подсказка (плейсхолдер в поле или всплывающая при наведении):	Видимость часто определяется правами, указать, для каких ролей делаем элемент видимым Если использован механизм тогглов, сделать описание
Доступность		disabled/enabled при каких условиях (например, кнопка отправки запроса на регистрацию задизейблена, пока не нажат чекбокс Согласия на обработку персданных)

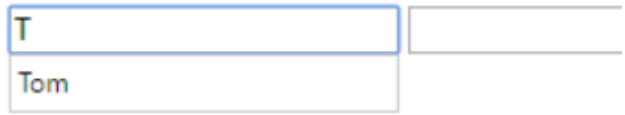
b. Алгоритм при нажатии:

9. Для текстового поля ввода такой:

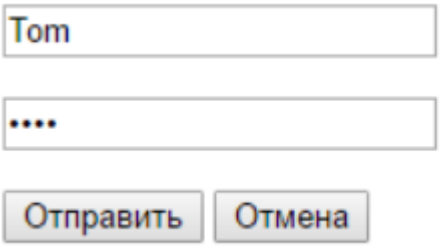
a. **Параметр**	**Значение**
Отображение	• Наименование: • Видимость: • Подсказка (плейсхолдер в поле или всплывающая при наведении): • Формат ввода:
Доступность	

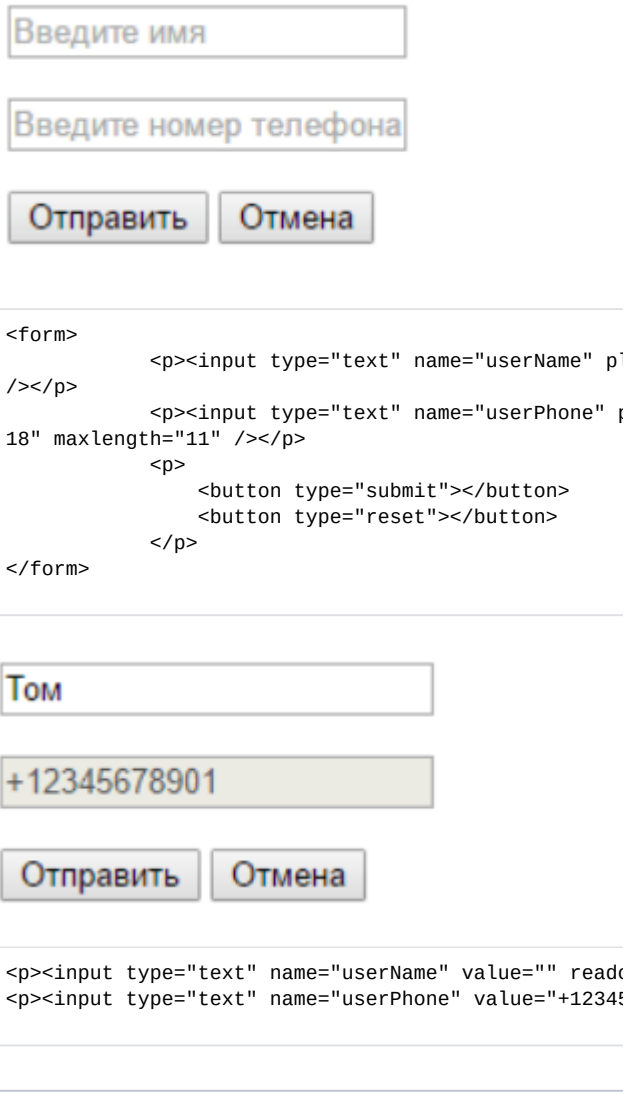
10. Важно описывать также обязательность массивов или объектов. Например, если по объекту не нашлось информации, то нужно или не присылать объект или присылать, но пустой, или присылать значение null вместо пустоты (нужно согласовать логику с разработчиками FE);
11. Придерживайтесь общепринятых терминов для описания элементов UI

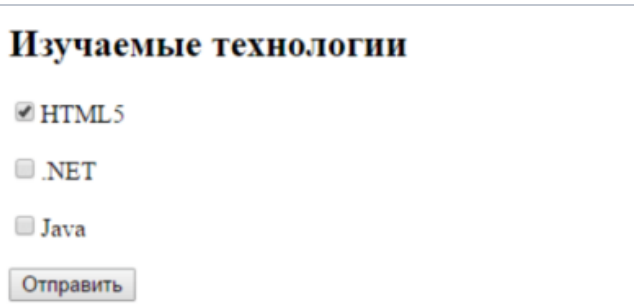
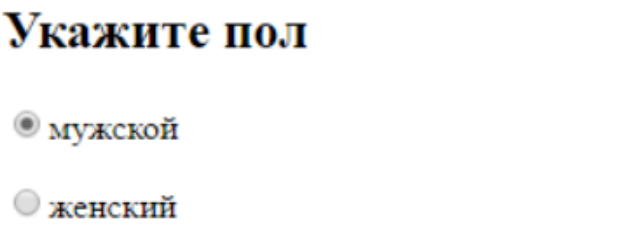
Элемент	Описание	Свойства	Атрибуты	Пример

Form	Формы **form** в html представляют один из способов для ввода и отправки данных.	Для настройки форм у элемента form определены следующие атрибуты:
		• `method`: устанавливает метод отправки данных на сервер. Допустимы два значения: `post` и `get`.
		Значение `post` позволяет передать данные на веб-сервер через специальные заголовки. А значение `get` позволяет передать данные через строку запроса.
		• `action`: устанавливает адрес, на который передаются данные формы
		• `enctype`: устанавливает тип передаваемых данных. Он свою очередь может принимать следующие значения:
		◦ `application/x-www-form-urlencoded`: кодировка отправляемых данных по умолчанию
		◦ `multipart/form-data`: эта кодировка применяется при отправке файлов
		◦ `text/plain`: эта кодировка применяется при отправке простой текстовой информации
		• `autocomplete ()` : позволяет запоминать вводимые данные и выводить список подсказок
				
				`<input name="login"/>`
				`<input type="submit" value="" />`
				 
				`<input name="login" />`
				`<input name="password" autocomplete="on" />`
				`<input type="submit" value="" />`

Элементы форм	Наиболее распространенным элементом ввода является элемент **input**. Однако реальное действие этого элемента зависит от того, какое значение установлено у его атрибута type.	• text: обычное текстовое поле • password: тоже текстовое поле, только вместо вводимых символов отображаются звездочки, поэтому в основном используется для ввода пароля • radio: радиокнопка или переключатель. Из группы радиокнопок можно выбрать только одну • checkbox: элемент флажок, который может находиться в отмеченном или неотмеченном состоянии • hidden: скрытое поле • submit: кнопка отправки формы • color: поле для ввода цвета • date: поле для ввода даты • datetime: поле для ввода даты и времени с учетом часового пояса • datetime-local: поле для ввода даты и времени без учета часового пояса • email: поле для ввода адреса электронной почты • month: поле для ввода года и месяца • number: поле для ввода чисел • range: ползунок для выбора числа из некоторого диапазона • tel: поле для ввода телефона • time: поле для ввода времени • week: поле для ввода года и недели • url: поле для ввода адреса url • file: поле для выбора отправляемого файла • image: создает кнопку в виде картинки Кроме элемента input в различных модификациях есть еще небольшой набор элементов, которые также можно использовать на форме: • button: создает кнопку • select: выпадающий список • label: создает метку, которая отображается рядом с полем ввода • textarea: многострочное текстовое поле	Атрибуты name и value У всех элементов ввода можно установить атрибуты `name` и `value`. Эти атрибуты имеют важное значение. По атрибуту `name` мы можем идентифицировать поле ввода, а атрибут `value` позволяет установить значение поля ввода	 `<input type="text" name="login" value="Tom"/>`

Кнопка Button	Кнопки представлены элементом **button**. Они обладают широкими возможностями по конфигурации. В зависимости от значения атрибута `type` мы можем создать различные типы кнопок
	
	Кроме элемента `button` для создания кнопок можно использовать элемент `input`, у которого атрибут равен `submit` или `reset`.
	
	Элемент input с атрибутом `type="image"` позволяет использовать в качестве кнопки изображение. Кроме наличия изображения, в остальном эта кнопка будет аналогична стандартной кнопке отправки `input type="submit"` или `button type="submit"`
		• `submit`: кнопка, используемая для отправки формы
		• `reset`: кнопка сброса значений формы
		• `button`: кнопка без какого-либо специального назначения
			Если кнопка используется для отправки формы, то есть у нее установлен атрибут `type="submit"`, то мы можем задать у нее ряд дополнительных атрибутов:
			
			• `form`: определяет форму, за которой закреплена кнопка отправки
			• `formaction`: устанавливает адрес, на который отправляется форма. Если у элемента form задан атрибут `action`, то он переопределяется
			• `formenctype`: устанавливает формат отправки данных. Если у элемента form установлен атрибут `enctype`, то он переопределяется
			• `formmethod`: устанавливает метод отправки формы (post или get). Если у элемента form установлен атрибут `method`, то он переопределяется
				
				
				`<form>`
				`<p><input type="text" name="login"/></p>`
				`<p><input type="password" name="password"`
				`<p>`
				`<button type="submit" formmethod="ge`
				`html"></button>`
				`<button type="reset"></button>`
				`</p>`
				`</form>`
				
				 
				
				`<!DOCTYPE html>`
				`<html>`
				`<head>`
				`<meta charset="utf-8">`
				`<title> HTML5</title>`
				`</head>`
				`<body>`
				`<form>`
				`<p>`
				`<input type="text" name="search" />`
				`<input type="image" src="search.png'`
				`</p>`
				`</form>`
				`</body>`
				`</html>`

Текстовые поля	Однострочное текстовое поле создается с помощью элемента input, когда его атрибут `type` и меет значение `text` `<input type="text" name="login" />`	С помощью ряда дополнительных атрибутов можно настроить текстовое поле: • dir: устанавливает направление текста • list: устанавливает список подсказок для ввода в поле • maxlength: максимально допустимое количество символов в текстовом поле • pattern: определяет шаблон, которому должен соответствовать вводимый текст • placeholder: устанавливает текст, который по умолчанию отображается в текстовом поле • readonly: делает текстовом поле доступным только для чтения • required: указывает, что текстовое поле обязательно должно иметь значение • size: устанавливает ширину текстового поля в видимых символах • value: устанавливает значение по умолчанию в текстовом поле Важно различать атрибуты `value` и `placeholder`, хотя оба устанавливают видимый текст в поле. Однако `placeholder` устанавливает своего рода подсказку или приглашение к вводу, поэтому он обычно отмечается серым цветом. В то время как значение `value` представляет введенный в поле текст по умолчанию Атрибуты `readonly` и `disabled` делают текстовое поле недоступным, однако сопровождаются разным визуальным эффектом. В случае с disabled текстовое поле затеняется Для создания полей поиска предназначен элемент input с атрибутом `type="search"` Для ввода пароля используется элемент input с атрибутом `type="password"`. Его отличительной чертой является то, что вводимые символы маскируются точками Для ввода чисел используется элемент `input` с атрибутом type="number". Он создает числовое поле, которое мы можем настроить с помощью следующих атрибутов: • `min`: минимально допустимое значение • `max`: максимально допустимое значение • `readonly`: доступно только для чтения • `required`: указывает, что данное поле обязательно должно иметь значение • `step`: значение, на которое будет увеличиваться число в поле • `value`: значение по умолчанию	

Флажки и переключатели	Флажок представляет элемент, который может находиться в двух состояниях: отмеченном и неотмеченном. Флажок создается с помощью элемента `input` с атрибутом type="checkbox"
	
	Переключатели или радиокнопки похожи на флажки, они также могут находиться в отмеченном или неотмеченном состоянии. Только для переключателей можно создать одну группу, в которой одновременно можно выбрать только один переключатель.
	
	Для создания радиокнопки надо указать атрибут `type="radio"`. И теперь другой атрибут `name` указывает не на имя элемента, а на имя группы, к которой принадлежит элемент-радиокнопка
				
				`<form>`
				`<p>`
				`<input type="checkbox" checked name=`
				`</p>`
				`<p>`
				`<input type="checkbox" name="dotnet"`
				`</p>`
				`<p>`
				`<input type="checkbox" name="java"/>`
				`</p>`
				`<p>`
				`<button type="submit"></button>`
				`</p>`
				`</form>`
				
				 
				`<form>`
				`<h2> </h2>`
				`<p>`
				`<input type="radio" value="man" chec`
				`</p>`
				`<p>`
				`<input type="radio" value="woman" na`
				`</p>`
				`</form>`

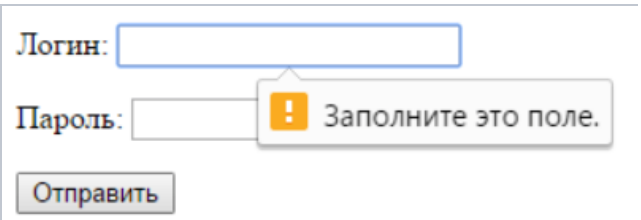
<table>
<tr>
<td>Валидации</td>
<td>Для создания валидации используется ряд атрибутов:

• required: требует обязательного ввода значения.
• min и max: минимально и максимально допустимые значения.
• pattern: задает шаблон, которому должны соответствовать вводимые данные.</td>
<td>Атрибут pattern

Атрибут pattern задает шаблон, которому должны соответствовать данные. Для определения шаблона используется язык так называемых регулярных выражений.

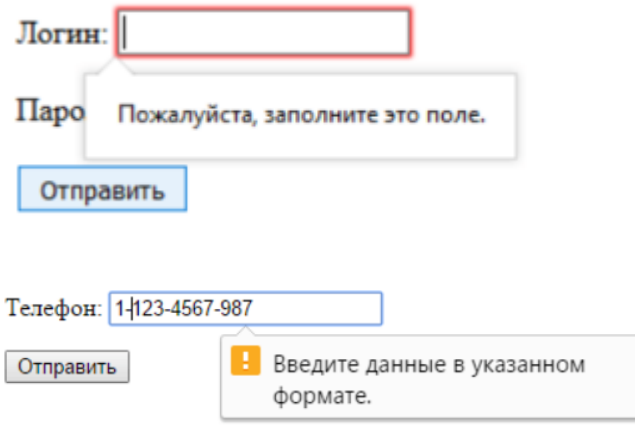
<pre><form method="get">
<p>
<label for="phone">:</label>
<input type="text" placeholder="+1-234-567-8901"

pattern="\+\d-\d{3}-\d{3}-\d{4}" id="phone" name="phone" />
</p>
<p>
<input type="submit" value="" />
</p>
</form></pre>
для ввода номера телефона используется регулярное выражение <code>\+\d-\d{3}-\d{3}-\d{4}</code>. Оно означает, что первым элементом в номере должен идти знак плюс +. Выражение \d представляет любую цифру от 0 до 9. Выражение <code>\d{3}</code> означает три подряд идущих цифры, а <code>\d{4}</code> - четыре цифры подряд. То есть это выражение будет соответствовать номеру телефона в формате "+1-234-567-8901".

Если мы введем данные, которые не соответствуют этому шаблону, и нажмем на отправку, то браузер отобразит ошибку</td>
<td></td>
<td> 

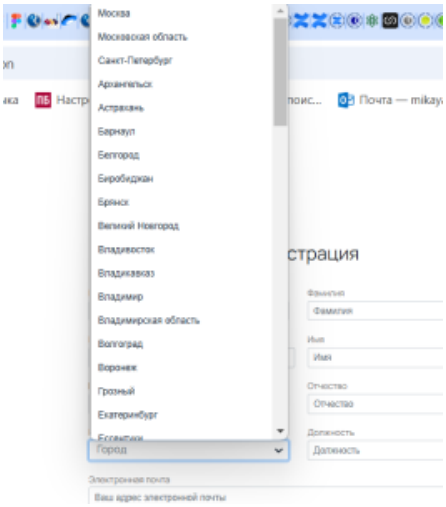
<pre><form method="get">
<p>
<label for="login">:</label>
<input type="text" required id="log
</p>
<p>
<label for="password">:</label>
<input type="password" required id='
password" />
</p>
<p>
<input type="submit" value="" />
</p>
</form></pre>

В зависимости от браузера визуализация сообщения может несколько отличаться

</td>
</tr>
</table>

12. Если предполагается адаптивный дизайн для функционала, описать требования к нему
13. Все текстовки (по возможности) должны приходить вместе с ответом по сетевому запросу. Должны возвращаться все необходимые тексты для формирования экрана, в некоторых случаях названия картинок). Такое поведение нужно реализовывать, чтобы минимизировать хранение бизнес-логики на FE.
    a. FE должен обладать минимальной бизнес-логикой. К примеру, в сценарии регистрации внешнего пользователя для выбора региона из списка используется захардкоженный на фронте список регионов, частично дублирующий данные из таблицы регионов АПР, в целевом решении список должен был бы получать FE от BE



   b. Обязательно описывать обработку ошибок и завершение процесса. Кейсы можно выводить в отдельные скриншоты в таблице скриншотов
   c. Если с BE приходит нечитаемая ошибка, предусмотреть юзер-френдли понятную и читаемую для пользователя текстовку на фронте (например, "Что-то пошло не так, мы уже работаем над этим" или "Ошибка получения данных, вернитесь на предыдущую страницу /обновите страницу").
14. ⚠️ Текстовки и макеты обязательно обсуждать и согласовывать с бизнес-заказчиком. Внешние пользователи системы не оценят элегантность архитектурного решения, но увидят плохо продуманный интерфейс. И это точно заметят бизнес-заказчики на демо. К ошибкам проектирования UI/UX относятся
   a. кнопки, которые слишком малы для точного нажатия;
   b. спрятанные кнопки, неочевидные для перехода;
   c. длинный скроллинг
   d. ссылки, расположенные слишком близко друг к другу;
   e. меню, в котором трудно ориентироваться;
   f. текст, который тяжело читать;
   g. всплывающие окна, раздражающие своей навязчивостью.
   h. отсутствие текстового комментария для заполнения поля ввода, если неочевидно, как заполнить это поле и т.д.
15. При постановке задачи на дизайнера обращать внимание на отрисовку негативных сценариев и ошибок.
16. Если экран ошибки типовой, сослаться на спецификацию, где он ранее описан
17. Учитывать комфортное для пользователя время ответа сервиса. Требование относится к нефункциональным, см. Раздел 4.3 - Требования к производительности
   a. Отклик в 100 миллисекунд воспринимается как мгновенный — так элементы интерфейса должны реагировать на наведение, движение мыши или клик.
   b. Задержка в 1 секунду заметна, но не выбивает пользователя из контекста. Нормально, если страница или список загружаются за секунду, но если секунда проходит между кликом и открытием меню, это медленно.
   c. Процессы, которые длятся больше секунды, нужно оптимизировать. Как вариант - отрисовывать скелетон страницы (реализован в VK)

3.2 Описание Backend

3.2.1 Интерфейсы передачи информации

1. При проектировании интерфейсов передачи информации укажите требования для функций взаимодействия, которые будут использоваться.
2. Определите соответствующие форматы сообщений.
3. Опишите особенности безопасности взаимодействия или шифрования, частоты передачи данных и механизмов синхронизации.

3.2.2 Шаблон сервиса

При создании главной страницы сервиса рекомендуется использовать Шаблон описания сервиса. Часть разделов описания заполняет разработчик бэкенда, часть сетевой администратор.

Структура описания:

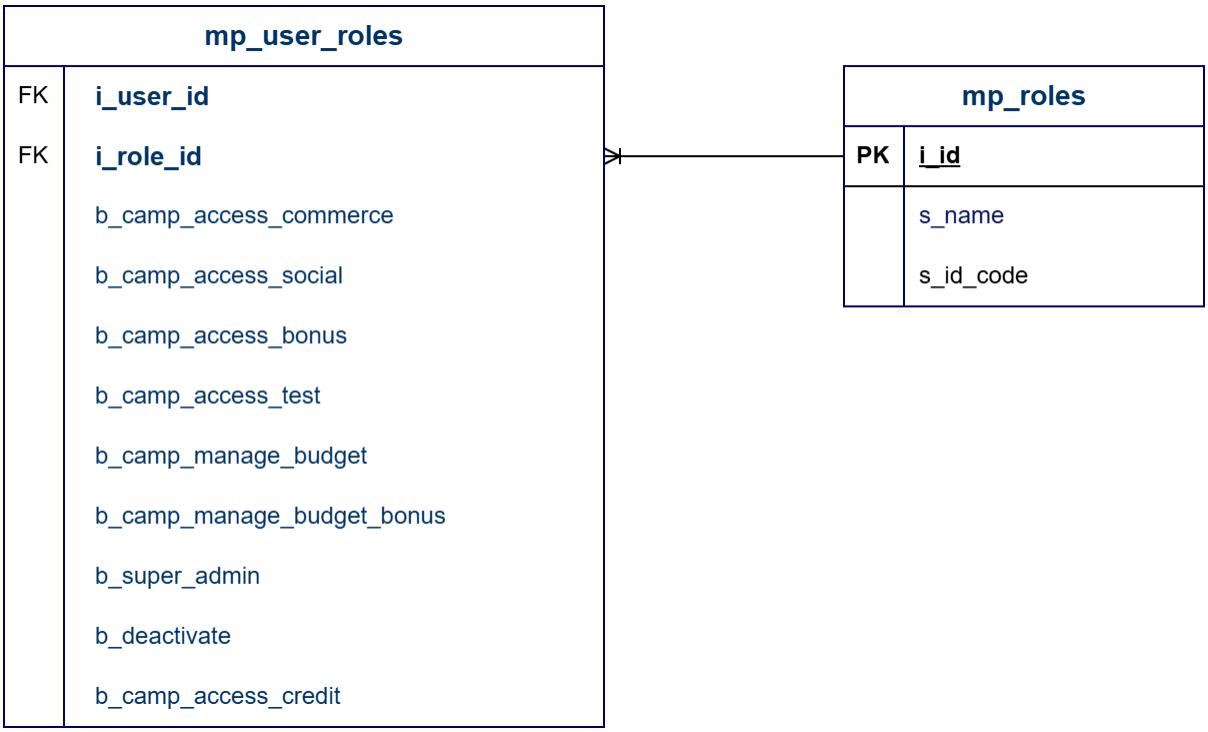
1. Описание
2. Ссылки
3. Требования к ресурсам
4. Отказоустойчивость
5. Авторизация
6. Описание окружений
   - Prod окружение
   - Stage окружение
   - Test окружение

7. Сборка
8. Деплой
9. Конфигурация
10. Переменные окружения для CI/CD
11. Использование ХП (можно добавлять в "описание окружений"), позволит тем, кто меняет структуры БД понять используется ли та или иная хранимка (и согласовать с руководителем проекта предстобщие изменениях)
12. Мониторинг
13. Сетевая связность
14. Доступы к сервису

Пример описания - ro-api

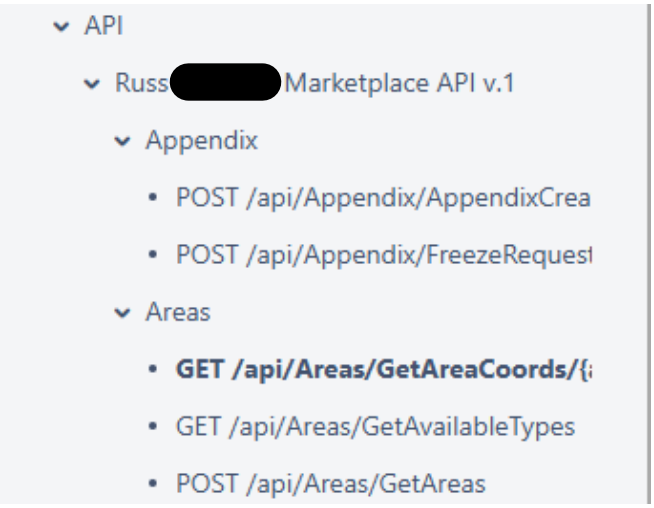
3.2.3 Модель данных. Описание БД

1. При описании БД требуется разделять типы БД.
2. MS SQL, Postrgres описываются отдельно в рамках своих страниц, исползуйте [PostgreSQL] Соглашение об именовании объектов базы данных;
3. Желательно все таблицы каждой БД описывать в одной статье;
4. Если таблиц много, то требуется создать ER-диаграмму со связями;
5. Если на диаграмме используются существующие в БД таблицы, цветом выделить новые таблицы;
6. Название полей должно строиться в нотации camel case (например order_id), см. детальнее в соглашении, ссылка из п.2.
7. В таблице с описанием важно указывать
    - обязательность
    - тип
    - ключи (PK, FK) и прочие необходимые характеристики.
8. Для таблиц планируется создать отдельный раздел [redacted]



3.2.4 Описание API

Методы API желательно описывать в подчиненной структуре. Родителем в данном случае считаем сервис.



При описании стоит не забыть о таких вещах, как:

- **Description** – короткое описание того, что делает API
- **Security Schemes** – используемый метод аутентификации (часто встречаемые API Keys, OAuth (2.0), OpenID Connect и HTTP Authentication Schemes – тут так же нужно указать какая именно схема используется.)
- **Transport security** – это может быть обычный открытый HTTP протокол или защищенный HTTPS.
- **Transport version** – HTTP был предложен Тимом Бернерсом-Ли еще в марте 1991-го и за это время вышло несколько обновлений. Соответственно, важно указать какую из версий вы будете использовать (например HTTP/1.0).
- **Endpoint** - не используйте глаголы в endpoint paths, дабы не перегружать его. О том, какое действие с точкой выполняет сервис лучше всего говорит метод, который используется в HTTP. Лучше использовать существительные, которые представляют собой названия конечной точки обрабатываемой интерфейсом (например GET /customer/order/{id} лучше, чем GET /customer/getOrder/{id}). Endpoint всегда зависит от многих факторов и лучше если на проекте определен единый подход к выбору его названия. Если такого нет, то можно этот момент обсудить с, например, с team lead или architect (в зависимости от вашего проекта и ролевой модели). Посоветуйтесь с разработчиком.
- **Method** – указывает какое действие должен выполнять сервис (например GET /PUT/ DELETE и т.д.). Детальнее об методах HTTP – тут.
- **Exchange pattern** – описание того, как именно должно происходить взаимодействие клиента с сервером (например Synchronous/Asynchronous Request – Response)
- **Headers** – помогают получить дополнительную информацию для управления взаимодействием с API. При этом, важно учесть, что есть обязательные и необязательные заголовки, в зависимости от выбранного HTTP -Method, а также продумать, какое значение должно приниматься по умолчанию, если необязательные Headers не указаны.
- **Request & response parameters –** описывая параметры, нужно не забыть указать:

    - Название параметра;
    - Тип данных;
    - Описание;
    - Наличие справочных значений (enum и его значения);
    - Обязательность параметра;

    - Поведение интерфейса, если не передан обязательный или необязательный параметр (для **request parameters**).
    - Поведение интерфейса, если не удалось получить/сгенерить обязательный параметр (для **response parameters**).

    - Требования к символам, которые необходимо использовать или не использовать.
    - Если значение числовое, требование к минимальному или максимальному значению.

В зависимости от выбранного типа метода параметры запроса могут передаваться как Path так и в body, поэтому их следует описывать отдельно (для body чаще всего используют JSON и XML формат).

- **Request and response examples** – в идеале, сделать несколько примеров на разные кейсы, при этом самое главное – увидеть полную структуру.
- **Errors** – условно можно поделить на:

    - полученные от системы источника данных (например, API не смогло подключиться к источнику из-за того, что он недоступен /источник вернул ошибку обработки данных);
    - ошибки API (любая логическая ошибка и т.п.);
    - ошибки подключения к серверу (когда запрос еще даже не дошел до API и где-то потерялся API Manager);

- **Caching** – если вы планируете реализовать кеширование, необходимо указать:

    - максимальное время актуальности данных в кеш;
    - логику удаления записи из кеша;
    - возможность принудительного обновления данных, которые хранятся в кеше;
    - триггер для актуализации данных в хранилище.

- **Performance** – тут могут быть разные метрики, например количество запросов в секунду (максимальное и среднее), настройки тайм-аут или пагинация.

Описание “подкапотной” части API

- **Diagram** – лучше всего использовать Sequence diagram, в которой очень удобно отобразить поведение и последовательность работы API.
- **Flow** – более детальное или расширенное описание того, что отображено в diagram (это опционально).
- **Mapping request & response parameters –** описывая маппинг нужно не забыть указать:
    - систему, которая является источником данных для каждого параметра (если у вас только один, это можно опустить);
    - тип данных параметра в системе источнике;
    - логику маппинга параметра/-ов на значение параметров API;
    - если параметр имеет справочное значение, необходим маппинг на справочники значения вашего API или, например, указать что вы пробрасываете то же значение, что получаете от источника)
    - обязательность параметра системы источника;
- поведение API, если не удалось получить у системы источника параметр, который является обязательным для вашего интерефейса.
- **Integration data** – тут должна быть указана необходимая информация для интеграции с системой источником. Данные которые должны быть указаны, зависят от того с кем происходит интеграция (БД, API, всевозможные файлы и тд).

Например, если это БД, тогда нужно указать название базы данных, к которой будет запрос, объект, из которого необходимо забирать данные, схему, где расположен объект. Так же должны присутствовать примеры запроса и ответа для интеграции .

- **Caching** – нужно ли хранить ответ от систем источника и логики с этим связанные

Метод <Наименование метода>

В качестве названия страницы указывается название метода API / Хранимой процедуры / HTTP-метод + URL.

Приложить ссылку на Swagger, если она известна.

История изменений

Дата	Версия	Описание изменения	Задача в JIRA	Автор

Описание

1. **Назначение**
    - Бизнес-описание логики работы метода (например, "Сохранение специального документа", "Отправка запроса на получение макета по id")
2. **Входные данные (Входящие параметры)**
    - Авторизация
        - Описание способа авторизации
    - Заголовки
        - Параметры path

Название	Описание	Тип данных	Обязательность

        - Параметры query

Название	Описание	Тип данных	Обязательность

    - Модель данных тела запроса

Поле	Описание	Тип данных	Обязательность	Варианты значений

        - если требуется, дополнительно укажите логику сопоставления, например, маппинг с полями таблиц БД
    - Пример запроса
        - В раздел необходимо приложить пример запроса, который сервис принимает на вход. Рекомендуется использовать макрос "Блок кода" или вложение к странице.
        - Допустимо не прикладывать пример запроса в случае, если сервис принимает на вход только авторизационные данные.
    - Кому разрешено вызывать метод (бывает полезно прописать, если есть требования по ограничению прав).
3. **Выходные данные (Исходящие параметры)**
    - Успешная обработка запроса
        - Коды ответа

HTTP Code	Описание
200	Запрос успешно обработан

- Заголовки
- Модель данных тела ответа

Поле	Описание	Тип данных	Обязательность

  - если требуется, дополнительно укажите логику сопоставления, например, маппинг с полями таблиц БД или ограничения поля
- Пример успешного ответа

- Неуспешная обработка запроса
  - Коды ответа

HTTP Code	Описание
401	Невалидный авторизационный токен
500	Внутренняя ошибка сервиса, исключение

  - **Статус-коды ответа HTTP:** указывают на результат выполнения запроса (например, 200 OK, 404 Not Found, 500 Internal Server Error). Полный перечень кодов HTTP тут.
  - Заголовки
  - Модель данных тела ответа

Поле	Описание	Тип данных	Обязательность

    - если требуется, дополнительно укажите логику сопоставления, например, маппинг с полями таблиц БД
  - Пример неуспешного ответа

4. **Описание алгоритма работы метода**
   - В разделе или на отдельной странице (в этом случае необходимо приложить перекрестную ссылку) описывается алгоритм работы метода.

     - Основной сценарий: описать сценарий работы метода (как и откуда получаем параметры запроса, каким образом сохраняем или переиспользуем ответ)
     - Сценарий исключений: описать сценарий обработки ошибок
     - Описать проверки и валидации, если есть
   - Допустимо и приветствуется использование средств визуализации шагов работы

5. **Взаимодействие с внешними компонентами. Источники данных**

3.3 Интеграции

3.3.1 Внешние интеграции

Если необходима интеграция с внешним провайдером данных, рисуется диаграмма последовательностей, описывается, какие данные мы забираем\поставляем на уровне вызовов. Описываются все схемы сообщений, триггеры отправки, частота, особенности взаимодействия (если есть).

3.3.2 Внутренние интеграции

Если сервис предоставляет данные вовне, в зависимости от способа интеграции делается либо swagger (в случае интеграции посредством передачи сообщений), либо же описываются сами структуры данных(если речь идет про шину данных), для ftp описывается структура файла и т д.

Также рисуется диаграмма последовательности для указания зависимостей визуализации взаимодействий других систем с описываемой.

3.4. Метрики

[redacted]

4 Нефункциональные требования

Нефункциональные требования представляют такие характеристики уровня производительности, как скорость реакции, плавность пользовательского интерфейса, безопасность и т.д. разрабатываемого приложения. Наиболее важные требования, которые почти всегда следует описать, это производительности, доступность и масштабируемость. Есть масса источников, в которых подробно описано, как рассчитывать эти показатели. Если речь идет про данные, следует учитывать юридические ограничения, конфиденциальность, время хранения

Согласно BABOK 3: Нефункциональные требования описывают атрибуты производительности или качества, которым должно соответствовать решение. Нефункциональные требования обычно измеримы и играют роль ограничений дизайна решения в целом.

4.1 Требования к безопасности

1. Укажите все требования, касающиеся защиты, целостности или конфиденциальности
2. Определите факторы, которые должны обеспечить защиту ПО от случайного или намеренного доступа, использования, идентификации или раскрытия:
    a. использование методов шифрования
    b. внедрение журналов данных/логирование
    c. ограничение связей и их уровня между функциями ПО
    d. проверка целостности данных для критических переменных
    e. при подготовке данного раздела необходимо согласование с представителями подразделения информационной безопасности

4.2 Требования к производительности

В разделе описываются требования к скорости выполнения различных операций в единицу времени и влияние на пользователя

4.2.1 Производительность операций пользовательского интерфейса

Система должна обеспечивать корректное время исполнения бизнес-процесса и корректное, комфортное для пользователя время обслуживания

№	Наименование операции	Уровень нагрузки в час пик

4.2.2 Производительность вспомогательных операций

В разделе описать уровень нагрузки на вспомогательный функционал (автоматические процессы без участия человека), в пределах которого система должна исполнять корректно бизнес-процесс

№	Наименование операции	Уровень нагрузки в час пик

4.2.3 Производительность вспомогательных операций

В разделе описать уровень нагрузки на вспомогательные системы (полностью автоматические процессы)

№	Наименование операции	Уровень нагрузки

4.2.4 Производительность внешних интерфейсов и информационных систем

Описать объем нагрузки, создаваемый информационной системой на внешние интерфейсы

№	Наименование операции	Максимальная нагрузка	Интеграция	Внешняя система

4.2.5 Время отклика операций пользовательского интерфейса

Описать требования ко времени отклика интерфейсных процедур

№	Наименование операции	Время отклика

4.2.6 Время обработки вспомогательных операций

Описать требования ко времени обработки данных для вспомогательных процессов и процедур

№	Наименование операции	Начало	Окончание	Длительность

4.2.7 Время обработки внешних интерфейсов и информационных систем

Описать требования к времени отклика внешних интерфейсов и внешних систем

№	Наименование операции	Начало	Окончание	Длительность

4.3 Требования к масштабируемости

Описать стандартные требования к масштабируемости

4.4 Требования к надежности

Описать требования

1. "мягкого старта" для систем, остановка которых влечет за собой повышенную нагрузку, например, одномоментный повторный вход пользователей после сбоя
2. к корректной работе приложений в случае отказа каких-либо компонент или сервисов
3. к резервированию аппаратного обеспечения и данных и восстановлению работоспособности информационной системы в виде дублирования компонент архитектуры (резервные сервера и т.д.)
4. касающиеся восстановления и защиты динамических и статических данных от логических и физических искажений
5. к контролю допустимых значений вводимых параметров пользовательского интерфейса (к этому разделу не относятся сценарии, когда пользователь может вызвать неработоспособность сервиса, запросив данные, например, за слишком большой период времени и т.д.)

4.5 Требования к сопровождению

4.5.1 Требования к резервированию и восстановлению

Описать требования к резервированию данных на жестких носителях (окно резервирования, требования к бекапированию и восстановлению данных

4.5.2 Требования к системе архивирования

Описать требования к процедуре архивирования устаревших данных

4.5.3 Требования к управлению доступом

Описать требования к управлению доступом и идентификации пользователей

4.5.4 Требования к протоколированию

Описать требования, связанные с протоколированием действий пользователей и системы, в т.ч. ошибок, исключений, возникающих в процессе эксплуатации системы

4.5.5 Требования к мониторингу информационной системы

Указываются требования по наличию встроенных средств определения состояния ИС и мониторинга ее ключевых метрик

4.5.6 Требования к подсистеме регистрации событий (аудит)

Если необходимо, по требования ИБ в информационной системе должен быть предусмотрен журнал регистрации событий и журнал изменения событий

4.5.7 Удаленный доступ к информационной системе

В разделе приводятся требования, предъявляемые к возможности получения удаленного доступа сотрудникам сопровождения